OctoBouncer

Dynamic Balancing and Trajectory Control System

Authors:

Oana Datcu
Faculty of Mechanics and Mechatronics

Ivan Alexandru
Faculty of Automatic Control and Computer Science

Coordinators:

Assoc. Prof. Dr. Eng. Daniel Rosner
Eng. Teodor Dicu
Teaching Assist. PhD Cand. Eng. Vlad Stănescu

Year 2026

Contents

1 Introduction **1**

2 Mechanical Design and Hardware Implementation **2**

2.1 Anatomy of the Actuation Arm and Kinematic Optimization 2
2.2 Chassis Integration and Component Management 3
2.3 Evolutionary Analysis of the Prototype 5
2.4 Electronics and Connectivity 5
2.4.1 Power Management 5
2.4.2 Actuation System and Motion Control 6
2.4.3 Communication and Data Flow 7
2.4.4 Electrical Schematic and Pin Assignment 7
2.4.5 Component Ratings 12

3 Mathematical and Kinematic Modeling **14**

3.1 Geometric Parameters of the System 14
3.2 Inverse Kinematics Calculation 14
3.2.1 Determining the Position of the Anchor Points 14
3.2.2 Solving the Arm Geometry (Law of Cosines) 15
3.3 Implementation in the Control Loop 15
3.4 Mechanical Limitations and Constraints 16

4 Control Strategy **17**

4.1 Distributed Control Architecture 17
4.2 PID Control Algorithm 17
4.3 Real-Time Image Processing Subsystem (Raspberry Pi) 18
4.3.1 Camera Calibration with Fisheye Lens 18
4.3.2 Ball Color Calibration (`1_tune_color.py`) 19
4.3.3 Processing Pipeline and State Estimation 19
4.3.4 Velocity Estimation and Two-Stage Filtering 20
4.3.5 Tuning Parameters 21
4.3.6 Live Visualization and Diagnostic Interface 21
4.3.7 Trajectory Generator and Pattern Selection 22
4.3.8 Real-Time PID Tuning Interface 23
4.3.9 Serial Communication Protocol 24
4.4 Data Filtering (EMA Filter) 25
4.5 Real-Time Management and Synchronization 25
4.6 Safety Limitations (Software Constraints) 25
4.7 Plate Height Control 25

5 Experimental Results and Conclusions **26**
5.1 Experimental Method 26
5.2 Trajectory Tracking 27
5.3 Settling Time and Steady-State Error 29
5.4 Effect of the Control Changes 31
5.5 Discussion and Limitations 32
5.6 Industrial and Commercial Applications 32
5.7 Conclusions and Future Developments 33

Chapter 1

Introduction

The problem of balancing a spherical body on a flat surface, known in the specialized literature as "Ball and Plate" [1], is one of the complex problems in the field of control systems engineering. Although at first glance it seems a simple problem, it introduces an unstable nonlinear dynamics, in which any positioning error is affected by external factors.

The **OctoBouncer** project, having as its source of inspiration the project of the same name [2], proposes a modern approach to this paradigm, extending the limits from simple stabilization to trajectory tracking, with the dynamic control of momentum (bouncing the ball) as the long-term goal. Our objective was not only to keep the ball at a fixed point, but to transform the platform into a system capable of complex interactions, such as complex trajectories.

This achievement required close collaboration between two fundamental disciplines:

- **Mechatronics:** Designing a platform driven by four arms, with three degrees of freedom (pitch, roll and height), characterized by low mass and high rigidity, capable of executing sudden vertical accelerations without compromising angular precision.
- **Automatic Control:** Developing a distributed control system, in which high-frequency image processing (Raspberry Pi 5 [3]) is closely tied to a real-time system (Teensy 4.1 [4]) in order to estimate the state of the ball and generate commands for the motors with minimal latency.

Through OctoBouncer, we demonstrate how feedback control algorithms and image processing can be integrated into a robust physical system, capable of holding the manipulated object at a fixed point and of moving it along predefined trajectories; managing its kinetic energy as well is the next step of the project.

Chapter 2

Mechanical Design and Hardware Implementation

The mechanical design of the **OctoBouncer** system evolved through an iterative process, with structural rigidity, efficient thermal management and easy maintenance as fundamental objectives. The chosen architecture is based on a redundant parallel robotic platform, a technical solution that offers a superior dynamic response compared to standard three-support-point configurations [13]. By using four independent actuation arms, the system manages to correct positioning errors in real time, ensuring increased stability of the platform plane even during translation movements, derived from the Stewart architecture [14].

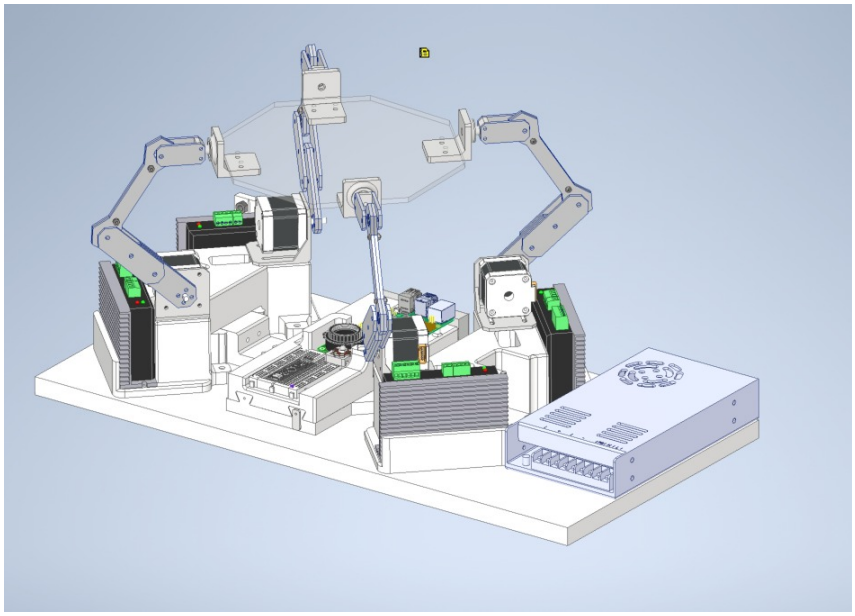


Figure 2.1: *Isometric representation of the complete OctoBouncer system in the final design stage.*

2.1 Anatomy of the Actuation Arm and Kinematic Optimization

The actuation arm is the most stressed component of the system, being designed to balance low mass with high mechanical strength. Its prototyping was carried out by 3D printing from **PET-G** [15], a material chosen for its superior impact resistance and

its ability to maintain tight assembly tolerances. Each arm integrates a complex hybrid structure: one active joint connected directly to the motor and three free joints.

From a technical standpoint, the elimination of mechanical backlash was achieved by integrating **precision bearings** at each pivot point. This decision was critical in preventing parasitic vibrations that could have destabilized the control algorithm at high frequencies. An important innovation compared to previous versions was the introduction of an additional joint on a perpendicular plane, positioned at the point of contact with the platform. This modification solved the diagonal mobility limitations, allowing the platform to execute complex trajectories without geometric constraints.

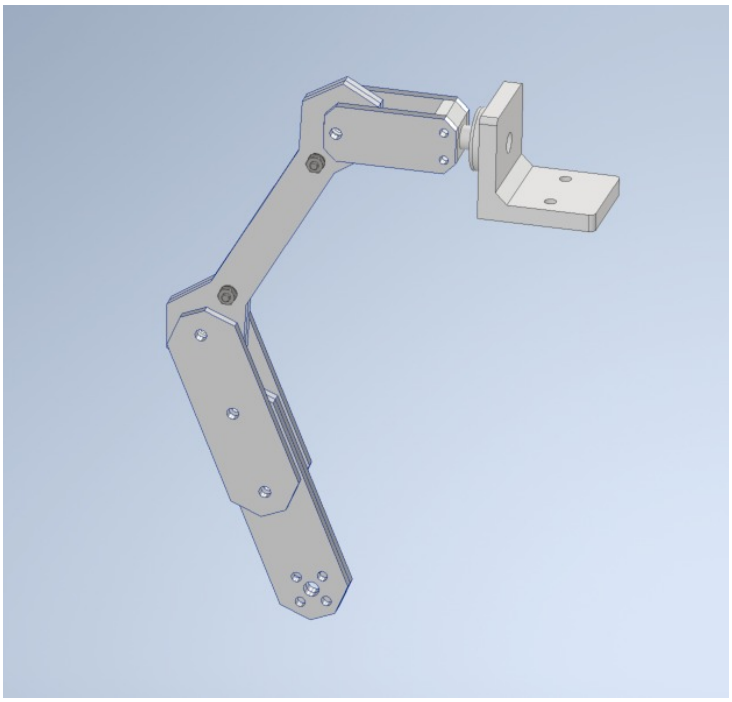


Figure 2.2: *Arm detail: optimization of the joints and the perpendicular pivot point for extended mobility.*

2.2 Chassis Integration and Component Management

The chassis of the system does not serve only as a structural support, but also as a casing and protection for the electronic components. The upper platform is made of **8 mm plexiglass**. We opted for this thickness because it offers optimal inertia for stability, while also having elastic properties that favor the rebound function of the ball.

Beyond the mechanical aspects, a major challenge was **thermal management**. Stepper motors tend to generate high temperatures during prolonged operation, which could lead to structural deformation of the polymer parts. To counter this phenomenon, we used metal brackets that absorb and dissipate the motors' heat, protecting the integrity of the PET-G base. Furthermore, the chassis design was provided with ventilation slots and dedicated air tunnels for the DM556 drivers [5], ensuring efficient active cooling under heavy loads.

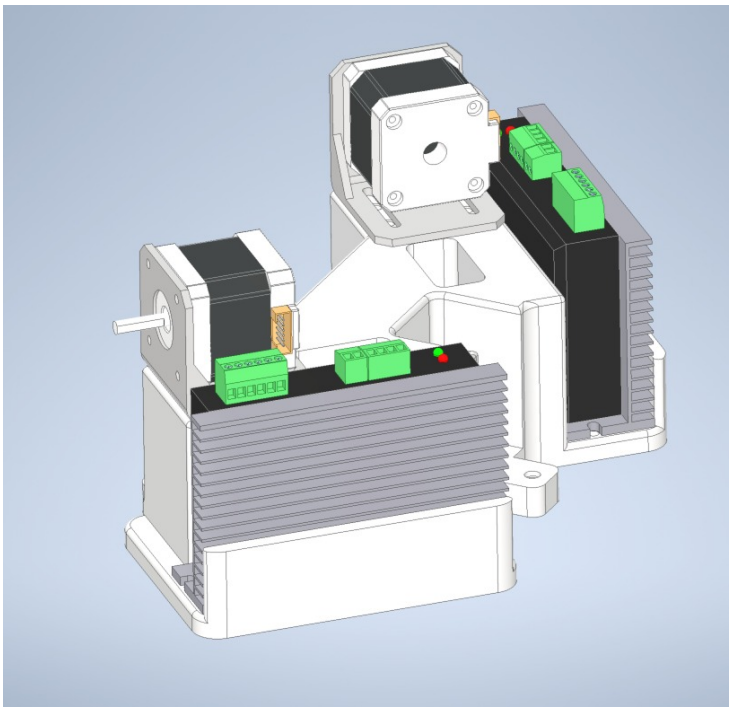


Figure 2.3: *Internal section: airflow architecture and ergonomic layout of the power components.*

Accessibility of the system was another priority in the design stage. We designed a **slider-type system** and interior channels for the efficient arrangement of the cables, facilitating quick access to the connections without the need to disassemble the arms. This approach reduces maintenance time and eliminates the risk of power or data wires interfering with the moving elements of the robot.

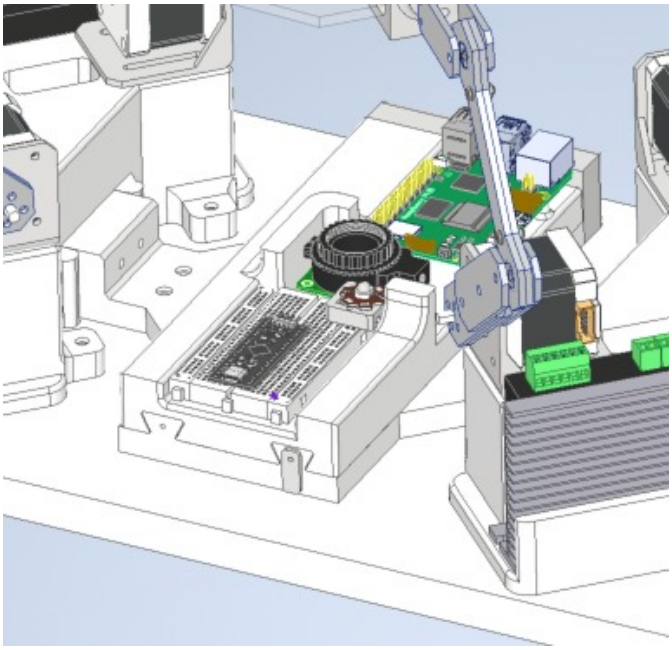


Figure 2.4: *Detail of the sliding mechanism designed for quick access to the central computing unit and wiring.*

2.3 Evolutionary Analysis of the Prototype

The transition from the initial version to the current **OctoBouncer** model involved a series of fundamental modifications. The first version suffered from low stability due to a center of mass that was too high and a lack of space for managing the wiring. The table below captures the technological evolution achieved between the two iterations.

Shortcomings of the Initial Version	**Solutions Implemented in the Final Model**
Robot too tall; instability under acceleration	Reduced profile to lower the center of mass
Difficult access to electronics; exposed wiring	Optimized chassis with slider system and protected channels
Thermal deformation of motor brackets	Integration of anti-deformation metal brackets
Limited mobility on certain axes	Addition of the perpendicular pivot joint

Table 2.1: *Summary of the optimizations made to the mechanical design.*

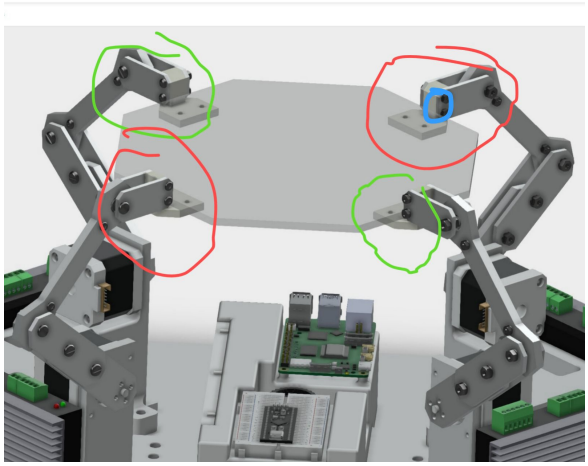


Figure 2.5: *The initial prototype.*

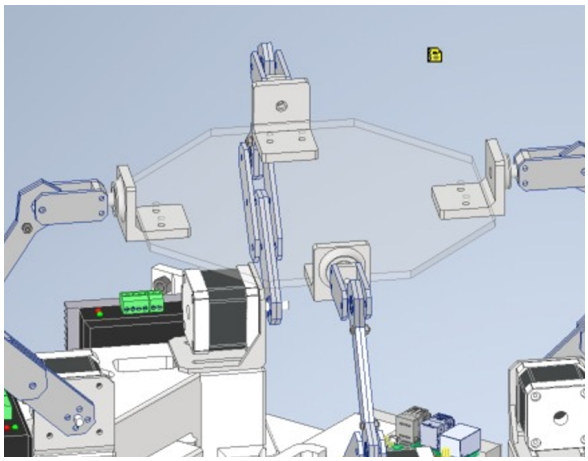


Figure 2.6: *The OctoBouncer system.*

2.4 Electronics and Connectivity

The electronic architecture of the **OctoBouncer** system was designed to ensure a minimal response time and high power supply stability, elements that are critical in the control of unstable systems. The system is built on a hierarchical structure, where the power supply for the high-power circuits and the logic one are separated in order to minimize electromagnetic interference (EMI).

2.4.1 Power Management

The main energy source is an efficient AC-DC unit, which provides a voltage of **48V**. This value was chosen to power the power stages of the 4 **DM556** drivers. The high

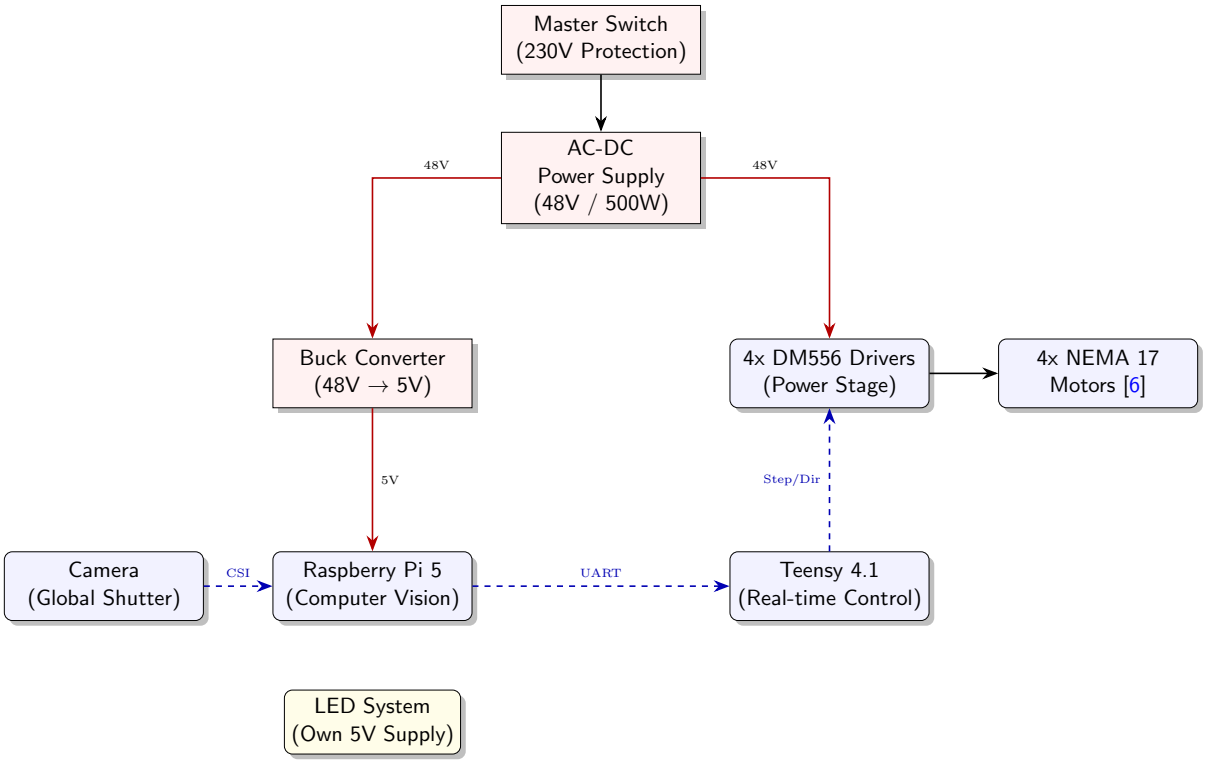


Figure 2.7: *Block diagram of the system.*

voltage allows the stepper motors to reach high angular speeds and sudden accelerations, necessary for the ball's "kick" phase, overcoming the torque limitations that usually appear at lower voltages (12V or 24V).

To power the logic subsystems, a **Step-down** converter lowers the voltage from 48V to **5V**, in order to power the **Raspberry Pi 5** and the camera. The DM556 drivers have no separate logic supply: their control electronics are powered from the 48V input.

The **Teensy 4.1** (the main microcontroller) is supplied through its USB connection to the Raspberry Pi 5, which is also used to upload the firmware. The **LED lighting** has its own 5V step-down module, independent of the Raspberry Pi 5, and is always on.

From the standpoint of safety during operation, the system integrates a **high-power load switch** placed directly on the input from the power supply. This switch fulfills the role of a safety barrier, allowing the instant disconnection of the entire assembly in the event of a mechanical or electrical failure, thus preventing damage to critical components or mechanical self-destruction in the event of a software error.

2.4.2 Actuation System and Motion Control

The control of the 4 stepper motors is carried out through industrial **DM556** drivers, using **Step/Dir**. The motion is smoothed by configuring the drivers to a high **microstepping** regime (1/16, corresponding to 3200 steps/revolution). This high resolution eliminates mechanical vibrations, giving the platform millimeter-level precision in controlling the tilt plane.

Although the source code includes support for closed-loop feedback through magnetic encoders, the current version of the system uses open-loop control. The absence of encoders simplifies the wiring and reduces the computational latency, relying on the extremely stable frequency of the signal generated by the Teensy 4.1 to prevent step loss.

2.4.3 Communication and Data Flow

The interfacing between the two computing units is important for the best possible performance of the system:

- **Camera interface:** The camera communicates with the Raspberry Pi 5 through the dedicated **CSI (Camera Serial Interface)** port, ensuring a high-bandwidth data transfer and zero latency in the GPU's processing buffer.
- **Host-to-Controller:** The Raspberry Pi 5 and the Teensy 4.1 communicate bidirectionally through the **UART** protocol at a speed of **115,200 baud**. To guarantee the correctness of the data in an environment with electrical noise induced by the motors, a **custom serial protocol** was implemented. It contains the ball's state vector (position and velocity on the 3 axes) in structured packets, validated through checksums, thus ensuring that no command is executed with corrupted data.

2.4.4 Electrical Schematic and Pin Assignment

Figures 2.8–2.11 show the complete electrical schematic, drawn in KiCad (the full sheet and the KiCad project are in the `hardware/schematic` folder of the repository). The pin numbers of the Raspberry Pi 5 are the physical pins of the 40-pin header; the pin numbers of the Teensy 4.1 are its digital pin numbers, as used in the firmware. The four motor channels are identical; the pins of each channel are given in Table 2.2.

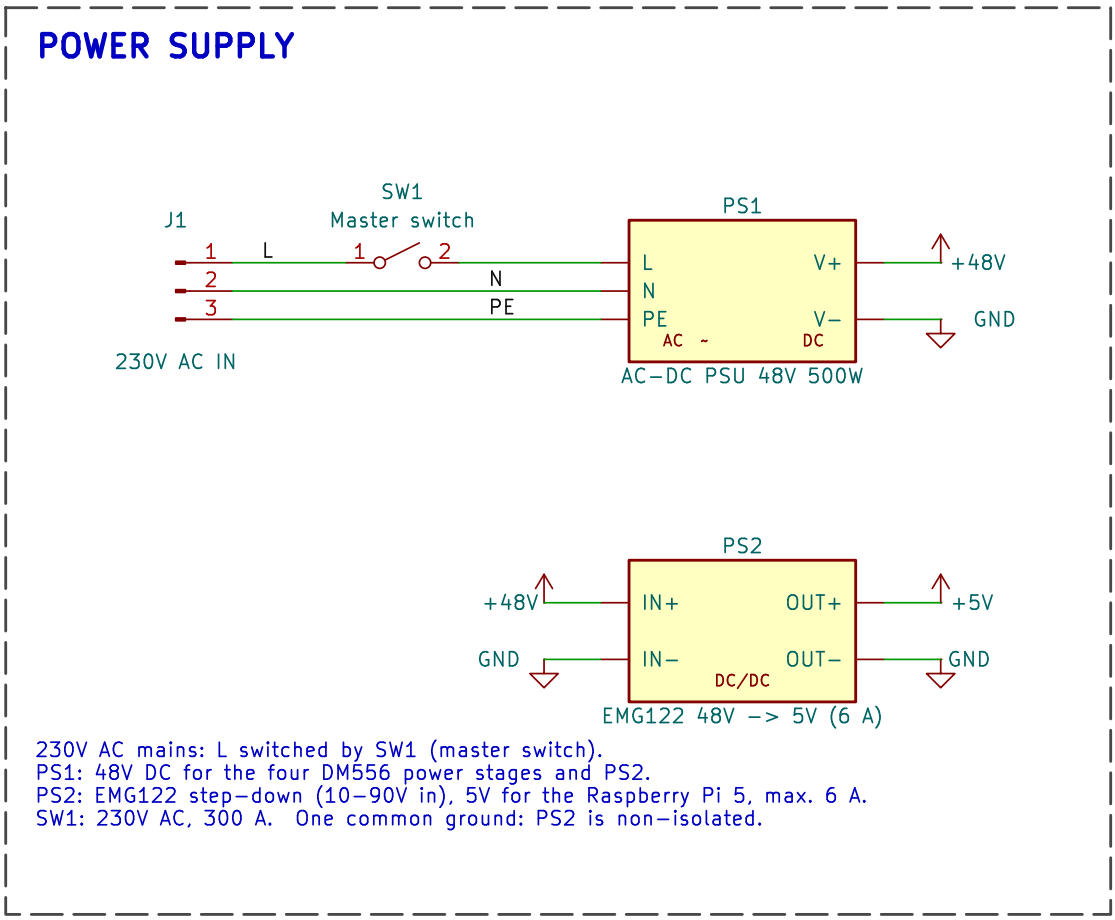


Figure 2.8: *Electrical schematic, part 1 of 4: power supply.*

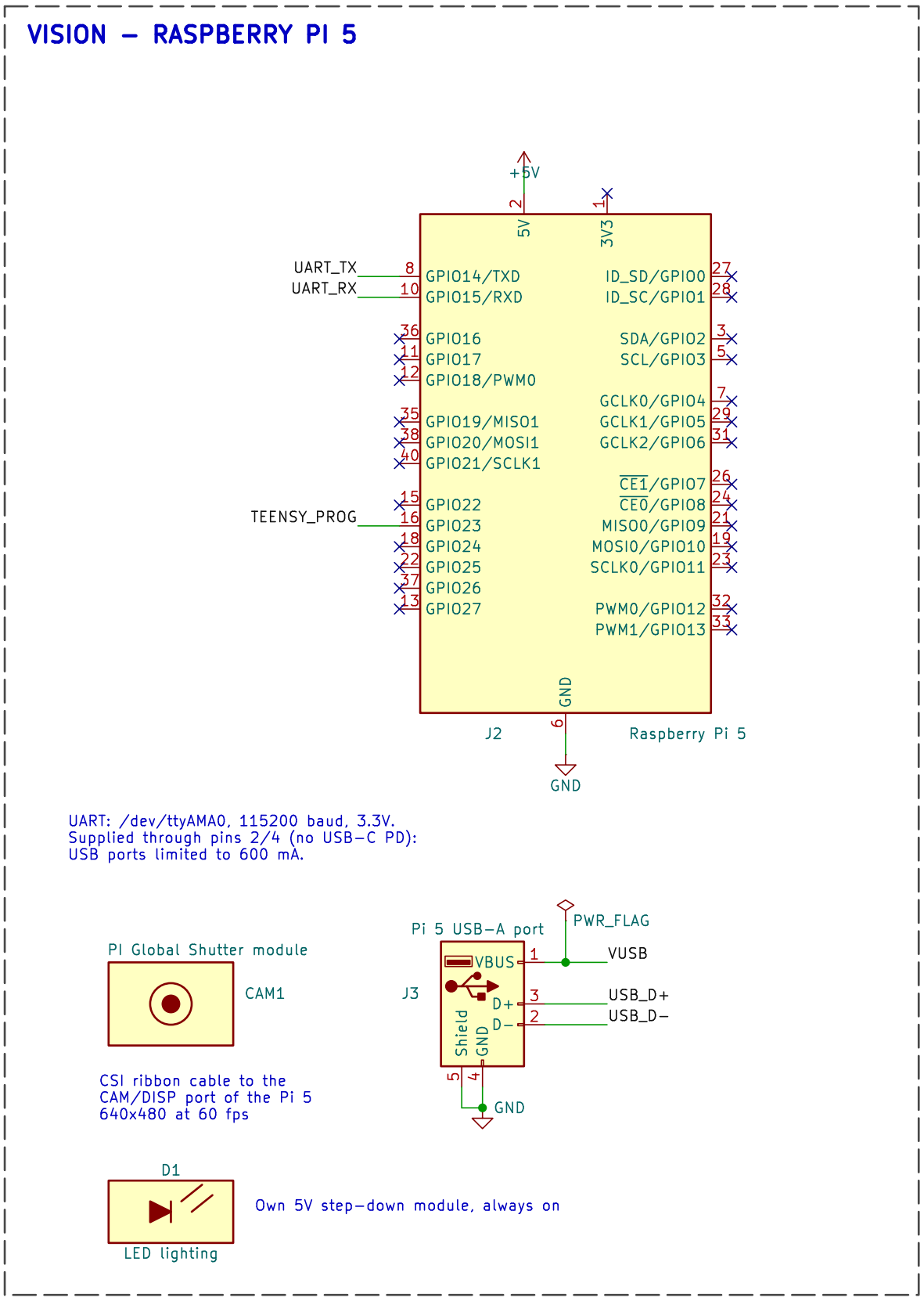


Figure 2.9: *Electrical schematic, part 2 of 4: Raspberry Pi 5, camera and lighting.*

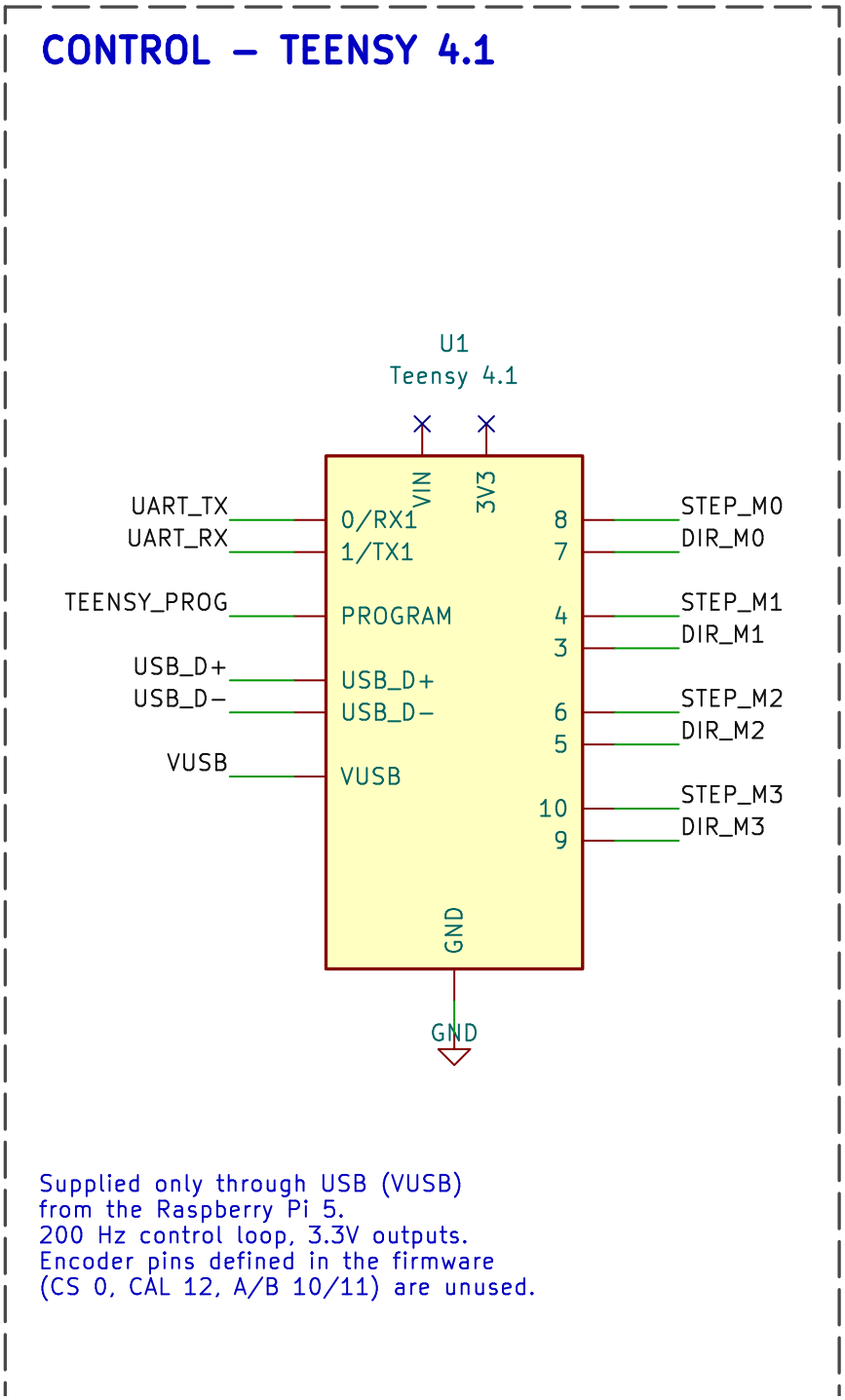


Figure 2.10: *Electrical schematic, part 3 of 4: Teensy 4.1.*

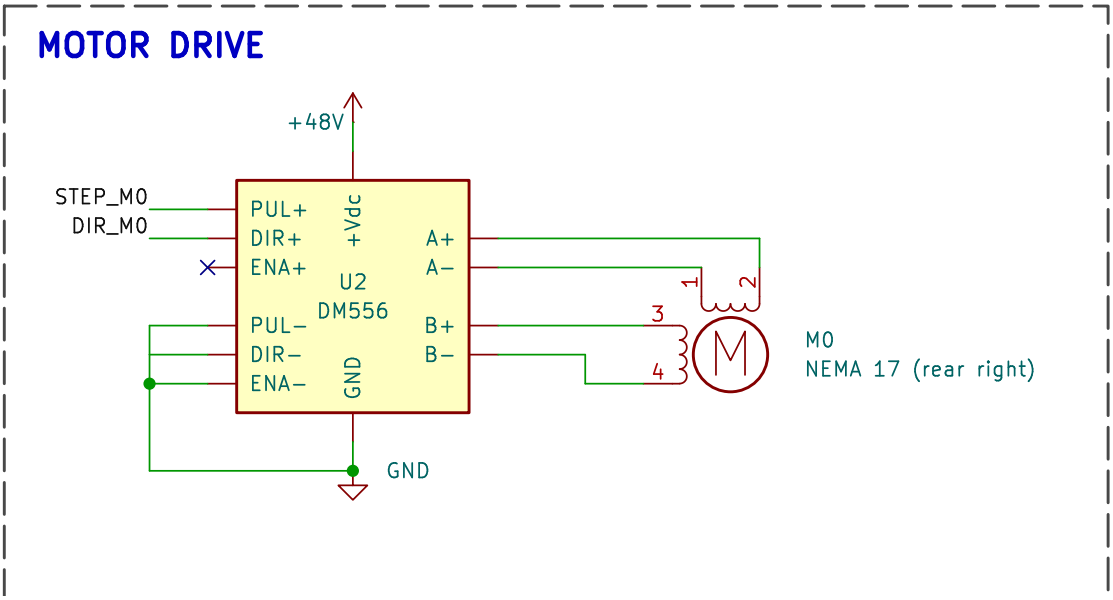


Figure 2.11: *Electrical schematic, part 4 of 4: motor drive, channel M0. The channels M1–M3 are identical; their STEP and DIR pins are listed in Table 2.2.*

Table 2.2 lists every connection of the schematic. The firmware also defines pins for the optional MT6835 magnetic encoders (CS on pin 0, CAL on pin 12, A/B on pins 10 and 11); since the encoders are not mounted in the current version, these definitions are inactive, and they would have to be moved to free pins before installing the encoders, because pin 0 is the UART receiver and pin 10 drives motor M3.

Signal	**From**	**To**	**Notes**
48V supply	PSU V+ / V−	DM556 +V / GND (all four), step-down IN+ / IN−	Power stage of the drivers
5V supply	Step-down OUT+ / OUT−	Pi 5 pins 2, 4 (5V) / pin 6 (GND)	Bypasses the USB-C input of the Pi
UART TX	Pi 5 pin 8 (GPIO14, TXD)	Teensy pin 0 (RX1)	`/dev/ttyAMA0` → `Serial1`, 115,200 baud, 3.3V
UART RX	Pi 5 pin 10 (GPIO15, RXD)	Teensy pin 1 (TX1)	Direction: Teensy → Pi
Flash request	Pi 5 pin 16 (GPIO23)	Teensy PROGRAM	Puts the Teensy in bootloader mode
USB	Pi 5 USB-A	Teensy micro-USB	Only power source of the Teensy (5V), common ground, firmware upload
Camera	Pi 5 CSI port	Camera module	Ribbon cable
Lighting	Separate 5V step-down module	LED system	Independent of the Raspberry Pi 5, always on
STEP M0 / DIR M0	Teensy pins 8 / 7	DM556 M0 PUL+ / DIR+	Motor M0, rear right
STEP M1 / DIR M1	Teensy pins 4 / 3	DM556 M1 PUL+ / DIR+	Motor M1, front right
STEP M2 / DIR M2	Teensy pins 6 / 5	DM556 M2 PUL+ / DIR+	Motor M2, front left
STEP M3 / DIR M3	Teensy pins 10 / 9	DM556 M3 PUL+ / DIR+	Motor M3, rear left
Signal return	DM556 GND terminal	PUL−, DIR− and ENA− of each driver	Common-cathode wiring; the return reaches the Teensy ground through the common ground of the non-isolated step-down converter
Enable	–	DM556 ENA+	Not connected (ENA− on GND): drivers always enabled
Motor phases	DM556 A+, A−, B+, B−	Motor coil A, coil B	One driver per motor

Table 2.2: *Pin assignment of the electrical connections.*

2.4.5 Component Ratings

Table 2.3 lists the electrical ratings of the components. The DM556 values are taken from the manufacturer's manual [5], the Teensy 4.1 values from the PJRC specifications [4], and the Raspberry Pi 5 values from the product brief [3].

Component	**Rating**	**Use in the system**
AC-DC PSU	Output 48V DC, 500 W (max. 10.4 A)	Supplies the four drivers and the step-down converter
Master switch	230V AC, 300 A, waterproof, ignition-protected	Disconnects the 230V AC input
Step-down converter	OEM EMG122 buck module: input 10–90V, output 1.5–60V (set to 5V), max. 6 A / 100 W, efficiency up to 97.5 %	Supplies the Raspberry Pi 5 (5 A recommended); non-isolated, so its input and output share the ground
Raspberry Pi 5	5V DC, 5 A recommended (27 W); GPIO 3.3V, not 5V tolerant	Image processing, UART master
Teensy 4.1	5V via USB, on-board 3.3V regulator; I/O 3.3V, not 5V tolerant	200 Hz control loop, STEP/DIR generation
DM556 driver (×4)	Supply 20–50V DC; output 1.8–5.6 A peak, set to 2.7 A RMS (3.8 A peak) with the DIP switches; isolated logic inputs, 5V nominal	Microstepping 1/16 (3200 pulses/revolution)
NEMA 17 motor (×4)	Bipolar, 1.8°/step (200 steps/revolution), rated 1.5 A per phase, 5 mm shaft [6]	Arm actuation
Camera	Raspberry Pi Global Shutter Camera (Sony IMX296 sensor) with fisheye lens, CSI, used at 640 × 480 and 60 fps	Ball detection
LED lighting	5V, from its own step-down module	Constant illumination, always on

Table 2.3: *Electrical ratings of the components.*

Three aspects of these ratings are relevant for the operation of the system:

- **Supply voltage margin:** The 48V bus is close to the 50V maximum of the DM556 drivers. During fast deceleration the motors return energy to the bus, so the voltage can briefly rise above the nominal value; the margin of 2V should be kept in mind if the power supply is replaced.
- **Logic levels:** The optically isolated inputs of the DM556 are specified for 5V signals, while the Teensy 4.1 outputs 3.3V. In the current system they are connected directly; a 3.3V → 5V level shifter would restore the nominal input current of the optocouplers.

- **Raspberry Pi 5 supply through the GPIO header:** Because the Pi 5 is supplied through pins 2 and 4, it does not negotiate its power with a USB-C supply, and by default it limits the total current of its USB ports to 600 mA. This is sufficient for the Teensy 4.1, which is the only USB device.

Chapter 3

Mathematical and Kinematic Modeling

The kinematic model of the **OctoBouncer** system has the role of establishing a precise correlation between the Cartesian coordinate space (the position and orientation of the platform) and the joint space (the rotation angles of the 4 stepper motors). For this project, we implemented an inverse kinematics model based on the geometry of parallel systems with 4 support points, using the mathematical basis specific to parallel robots [13], derived from the architecture of the Stewart manipulator [14].

3.1 Geometric Parameters of the System

In the geometric calculations, we used the following constant dimensions, determined through CAD design and direct measurements:

- L_1 **(98.1 mm):** Length of the arm fixed on the motor shaft.
- L_2 **(92.0 mm):** Length of the transmission rod (intermediate link).
- L_3 **(54.0 mm):** Vertical offset created by the joint under the platform.
- R_b **(129.0 mm):** Radius of the fixed base (distance from the center to the motor axis).
- R_p **(110.0 mm):** Radius of the mobile platform (distance from the center to the spherical joint).
- h_0 **(100.0 mm):** Neutral operating height of the platform.

3.2 Inverse Kinematics Calculation

The purpose of the algorithm is to determine the motor angle α_i for each arm $i \in \{0, 1, 2, 3\}$, starting from the orientation angles θ_{pitch} (pitch) and θ_{roll} (roll).

3.2.1 Determining the Position of the Anchor Points

Each motor is positioned at an angle γ_i relative to the central axis, defined in the source code as:

$$\gamma = \{225^\circ, 135^\circ, 45^\circ, 315^\circ\} \tag{3.1}$$

For a given orientation of the platform, the vertical height Z_i of each joint on the platform is calculated by projecting the tilt angles onto the radial geometry:

$$Z_i = h_0 + R_p \cdot (\sin(\theta_{roll}) \cdot \cos(\gamma_i) + \sin(\theta_{pitch}) \cdot \sin(\gamma_i)) \tag{3.2}$$

Taking into account the length of the joint L_3, the vertical target for the transmission rod becomes $Z_{target} = Z_i - L_3$.

3.2.2 Solving the Arm Geometry (Law of Cosines)

Solving the inverse kinematics for parallel manipulators is done by decomposing the system into independent kinematic loops (closed-loop vector equations) [13]. The problem reduces to a triangle formed in the vertical plane of each motor. The radial distance between the point on the base and the one on the platform is constant in horizontal projection:

$$X_{offset} = |R_b - R_p| \tag{3.3}$$

The direct distance D between the motor axis and the spherical joint on the platform is:

$$D = \sqrt{X_{offset}^2 + Z_{target}^2} \tag{3.4}$$

Applying the **Law of Cosines** in the triangle formed by L_1, L_2 and D, we obtain the interior angle β:

$$\beta = \arccos\left(\frac{L_1^2 + D^2 - L_2^2}{2 \cdot L_1 \cdot D}\right) \tag{3.5}$$

The final rotation angle of the motor, α_i, is the sum of the base geometric angle and the correction β:

$$\alpha_i = \arctan 2(Z_{target}, X_{offset}) + \beta \tag{3.6}$$

3.3 Implementation in the Control Loop

The complete model above is implemented in the firmware (the `calculateArmAngle` function) and describes the geometry of the mechanism. In the current version of the real-time loop, however, it is not called: for the small tilts needed for balancing, the relation between the tilt of the platform and the rotation of each arm is close to linear, so the controller uses a **linear mixing** of the two tilt commands. The PID outputs θ_{pitch} and θ_{roll} (Chapter 4) are expressed directly in degrees of arm rotation, and each arm receives:

$$\alpha_i = \alpha_h + \sigma_i^p \, \theta_{pitch} + \sigma_i^r \, \theta_{roll} \tag{3.7}$$

with the signs (σ_i^p, σ_i^r) equal to $(+1, +1)$, $(-1, +1)$, $(-1, -1)$ and $(+1, -1)$ for the motors M0 (rear right), M1 (front right), M2 (front left) and M3 (rear left), where α_h is the common arm angle that sets the height of the platform (Section 4.7). A positive pitch command thus raises the rear arms and lowers the front ones, and a positive roll command raises the right side. All angles α_i are measured from the mechanical low stop of the arms. This simplification avoids the trigonometric calculations in the 200 Hz loop; the gains of the PID controller absorb the constant ratio between arm rotation and platform tilt.

3.4 Mechanical Limitations and Constraints

For reasons of structural safety and to prevent collisions between links or the locking of joints (singularities), we implemented a strict software limitation of the tilt commands:

$$-10^\circ \leq \theta_{pitch,roll} \leq 15^\circ \tag{3.8}$$

In addition, every arm command is limited to the range $5^\circ \leq \alpha_i \leq 40^\circ$ above the low stop, so that the arms never press against the stop, and the common height angle α_h is kept between 15° and 30°, which leaves 10° of tilt range on both sides.

These constraints keep the system in a zone of relative linearity and prevent the motors from being forced into extreme positions. At the moment, the *bouncing* function is disabled in order to protect the integrity of the rod-joint assembly under heavy dynamic loads, as it has not yet been tested sufficiently.

Chapter 4

Control Strategy

The control system of the **OctoBouncer** is distributed between two processing units, the final goal being to keep the ball in the center of the platform (or to follow a trajectory) by continuously adjusting the tilt plane.

4.1 Distributed Control Architecture

To maximize performance, we separated the processing tasks into two levels:

1. **High Level:** The Raspberry Pi 5 runs the image processing algorithms under OpenCV, extracting the (x, y) coordinates of the ball and calculating the velocity vectors (V_x, V_y) at a high frequency.
2. **Execution Level:** The Teensy 4.1 receives the data through UART and runs the PID control loop at a frequency of **200 Hz** (5 ms), generating the Step/Dir signals for the motors.

4.2 PID Control Algorithm

The core of the stabilization is a **Proportional-Integral-Derivative (PID)** controller [11]. The calculation formula for the desired tilt angle (on each axis) is:

$$\theta(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau)d\tau + K_d \cdot (V_{ball}(t) - V_{target}(t)) \tag{4.1}$$

where $e(t)$ is the difference between the filtered position of the ball and the target position.

Where the components fulfill the following roles:

- **Proportional Term (K_p):** Reacts instantaneously to the ball's position error relative to the target (the center of the platform or the current point of the trajectory).
- **Integral Term (K_i):** Eliminates steady-state errors caused by possible mechanical imperfections or residual tilts of the base. We implemented an *anti-windup* limitation of the integral to prevent saturation of the integrator.
- **Derivative Term (K_d):** Acts as a "brake" (damping). The velocities of the ball and of the target are estimated on the Teensy by differentiating their EMA-filtered positions (Section 4.4) and smoothing the result with a second EMA filter ($\alpha = 0.2$), which attenuates the numerical noise of the differentiation [12]. The term acts on the velocity of the ball *relative to the target*: a ball that moves together with a moving target is not braked, while with a fixed target (CENTER mode) the term reduces

to a derivative on the measurement, which avoids command jumps when the target changes. The velocity estimated by the Raspberry Pi is also transmitted, but is not used by the controller. Section 5.4 compares this form with a derivative acting on the ball's own velocity.

4.3 Real-Time Image Processing Subsystem (Raspberry Pi)

The high level of the control architecture is implemented as a Python script running continuously on the **Raspberry Pi 5**, which can be launched automatically at startup. Its responsibility is to transform the stream of frames from the camera into a complete state vector of the ball, made up of three-dimensional position and velocities, transmitted in real time to the Teensy 4.1. The subsystem code is organized modularly, each script fulfilling a distinct role in the development and operation flow of the system.

4.3.1 Camera Calibration with Fisheye Lens

The camera used is a **Raspberry Pi Global Shutter Camera**: its sensor exposes all pixels at the same time, so a fast-moving ball is not distorted as with a rolling shutter. It is equipped with a **fisheye (ultra-wide angle)** lens, which, although it maximizes the field of view in order to cover the entire surface of the platform, introduces a pronounced radial distortion. Straight lines appear curved, and objects at the edge of the frame are scaled non-uniformly relative to the center. Using raw pixel coordinates in the position calculations would introduce unacceptable systematic errors into the control loop.

The calibration process takes place in two steps implemented in the `camera_calib` module:

1. **Capturing the calibration images (`1_capture_images.py`):** A checkerboard pattern with a grid of 9×7 squares (8×6 interior corners) is used. The operator presents the board in front of the camera at various angles and distances, saving one frame at each press of the `'s'` key, at the moment when the `findChessboardCorners` algorithm [16] visually confirms the complete detection. The calibration set used comprises 31 images, covering sufficient spatial variations to characterize the distortion across the entire sensor area.

2. **Calculating the intrinsic parameters (`2_calibrate_calc.py`):** Based on the set of images, the `cv2.fisheye.calibrate` algorithm from OpenCV [7] is run, which determines the camera's intrinsic matrix $\mathbf{K}$ and the fisheye distortion vector [8, 9] $\mathbf{D}$ (with 4 coefficients). The correction of the corners is carried out through `cornerSubPix` to a precision of 0.001 pixels. The resulting parameters are saved in a binary Numpy array file, `cam_calib_fisheye.npz`, loaded at every system startup.

Based on these parameters, two rectification tables (`map1`, `map2`) are generated through the `cv2.fisheye.initUndistortRectifyMap` function. The `cv2.remap` function applies a bilinear rectification on each captured frame, efficiently transforming the initial fisheye projection into a distortion-free plane-perspective image. The parameter `balance = 1.0` ensures that the entire field of view is preserved without cropping, at the cost of some black edges at the corners.

4.3.2 Ball Color Calibration (`1_tune_color.py`)

The detection of the ball is based on color segmentation in the **HSV** space [10], chosen instead of RGB for decoupling the hue information from lighting variations. Before the first use, or whenever the ball, the light source or the ambient environment changes, the interactive color calibration script must be run.

The graphical interface exposes six sliders, corresponding to the lower thresholds $[H_{min}, S_{min}, V_{min}]$ and to the upper thresholds $[H_{max}, S_{max}, V_{max}]$. The values are adjusted in real time while simultaneously observing the original image and the resulting binary mask, with the aim of maximizing the white area corresponding to the ball and eliminating any false positives from the background. The camera is configured in **manual** mode (without auto-exposure and without automatic white balance), with the exposure time set to **20 ms** (limited by the camera to the frame period of 16.7 ms at 60 fps) and the analog gain at **2.0**. Upon pressing the `'q'` key, the calibrated values are saved in the `hsv_values.txt` file and will be loaded automatically at subsequent startups.

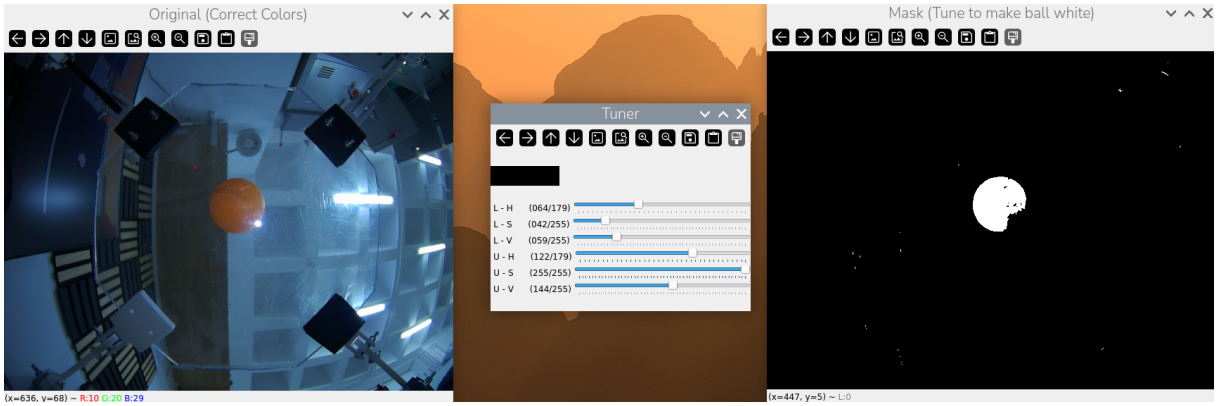


Figure 4.1: *The HSV calibration interface: the original window (left), the slider panel with the current thresholds (center) and the resulting binary mask - the ball appears white, the background black (right).*

4.3.3 Processing Pipeline and State Estimation

The main processing loop, implemented in `4_headless_track.py` (without display, for autonomous operation) and `4_gui_track.py` (with display, for monitoring and diagnostics), successively goes through the stages described below at an acquisition frequency of **60 fps**, a resolution of 640×480 pixels, through the dedicated **CSI** interface of the Raspberry Pi 5.

1. **Capture and geometric correction:** The BGR frame is captured directly through the `Picamera2` library [17] in native format, avoiding an additional color conversion. The precomputed fisheye rectification is applied immediately.

2. **HSV segmentation and morphological operations:** The corrected image is converted into the HSV space, and the binary mask is obtained through a color threshold. An **erosion** followed by a **dilation** with a 3×3 kernel is applied to the mask, eliminating point artifacts without affecting the geometry of the ball.

3. **Contour selection and radius estimation:** The `cv2.findContours` function extracts all the connected regions from the mask. The contour with the maximum

area is selected as the sole candidate. The `cv2.minEnclosingCircle` function determines the center (x_{px}, y_{px}) and the radius r_{px} in pixels. A minimum threshold filter $r_{px} > 5$ eliminates false detections caused by reflections or other surrounding objects with similar hues.

4. **3D reconstruction through the corrected pinhole model:** The depth Z is estimated from the apparent diameter of the ball in the geometrically corrected image, using an extended pinhole model with angular correction to compensate for the perspective elongation effect at the edges of the image:

$$Z_{pinhole} = \frac{f_x \cdot D_{real}}{2 \cdot r_{px}}, \quad Z_{final} = Z_{pinhole} \cdot \sqrt{1 + \left(\frac{d_{center}}{f_x}\right)^2} \tag{4.2}$$

where $D_{real} = 40\,\text{mm}$ is the real diameter of the ball, f_x is the focal length extracted from the new camera matrix $\mathbf{K}'$, and d_{center} is the distance in pixels from the optical center. The planar coordinates in millimeters are calculated through inverse projection:

$$X_{real} = \frac{(x_{px} - c_x) \cdot Z_{final}}{f_x}, \quad Y_{real} = \frac{(y_{px} - c_y) \cdot Z_{final}}{f_y} \tag{4.3}$$

5. **Median filtering of the depth:** Because X_{real} and Y_{real} are proportional to Z_{final}, a single-frame error in the estimated radius would move the ball position and produce a false velocity. The depth is therefore passed through a **median filter**: the median of the last 15 frames (0.25 s) is used to scale X and Y, and the median of the last 5 frames is reported as Z. The median removes isolated outliers without smoothing real movements as strongly as an average.

4.3.4 Velocity Estimation and Two-Stage Filtering

Since the velocity of the ball cannot be obtained through direct measurement, it is estimated numerically. Differentiating the positions between consecutive frames amplifies the quantization errors, requiring a two-level filtering for noise attenuation:

1. **Filtering the raw position (EMA with $\alpha_{pos} = 0.6$):** Before computing the derivative, the raw positions are filtered. A higher α means that the system tracks the ball quickly, but passes more noise to the velocity estimation.

2. **Filtering the calculated velocity (EMA with $\alpha_{vel} = 0.15$):** The resulting numerical velocity is passed through an aggressive low-pass filter. The value $\alpha_{vel} = 0.15$ was determined experimentally: higher values (> 0.4) leave residual vibrations that destabilize the derivative term of the PID, while lower values (< 0.05) introduce a noticeable phase delay at sudden changes of direction.

These velocities are displayed in the diagnostic interface and transmitted to the Teensy, which computes its own estimate for the controller (Chapter 4). An additional **deadband** mechanism (1.5 mm) treats displacements that are below the threshold as null, eliminating false velocities under stationary-ball conditions. If the ball disappears from the field of view, the velocity is decremented exponentially (factor 0.8 per frame) instead of being reset abruptly, preventing command shocks at re-detection.

4.3.5 Tuning Parameters

The optimal functioning of the frame capture and processing subsystem depends on the correct calibration of the parameters grouped in the following table. They are adjustable both by directly modifying the source files and at runtime through the dedicated graphical interfaces.

Parameter	**Current value**	**Impact when adjusted**
$[H_{min}, S_{min}, V_{min}]$	$[98, 145, 40]$	Too narrow: detection losses; too wide: false positives from the environment
$[H_{max}, S_{max}, V_{max}]$	$[116, 255, 255]$	Upper limit of the accepted color; S_{max} and V_{max} at 255 include the entire brightness range
$r_{px,min}$ (minimum radius)	5 px	Below this value, the detection is ignored; prevents false positives from reflections
α_{pos} (position EMA)	0.6	Higher values: fast tracking, but more noise toward the velocity estimator
α_{vel} (velocity EMA)	0.15	Compromise between attenuating the derivative noise and the response latency at impact
Deadband	1.5 mm	Eliminates velocity drift at rest; too large masks real slow movements
`balance` (undistort)	1.0	0.0 = central crop without black edges; 1.0 = maximum field of view preserved

Table 4.1: *Configuration parameters of the visual detection subsystem and their effect on the behavior of the system.*

4.3.6 Live Visualization and Diagnostic Interface

The versions with display (`4_gui_track.py`, `4_gui_track_w_mode.py`) overlay a diagnostic layer on top of the corrected video stream, intended exclusively for tuning and debugging. The overlay displays the ball detection status (FOUND/LOST), the spatial coordinates (X, Y, Z) and the velocities (V_x, V_y, V_z) in real time, the green circle superimposed on the detected ball, the current operating mode of the system, as well as a real-time animation in the bottom-right corner.

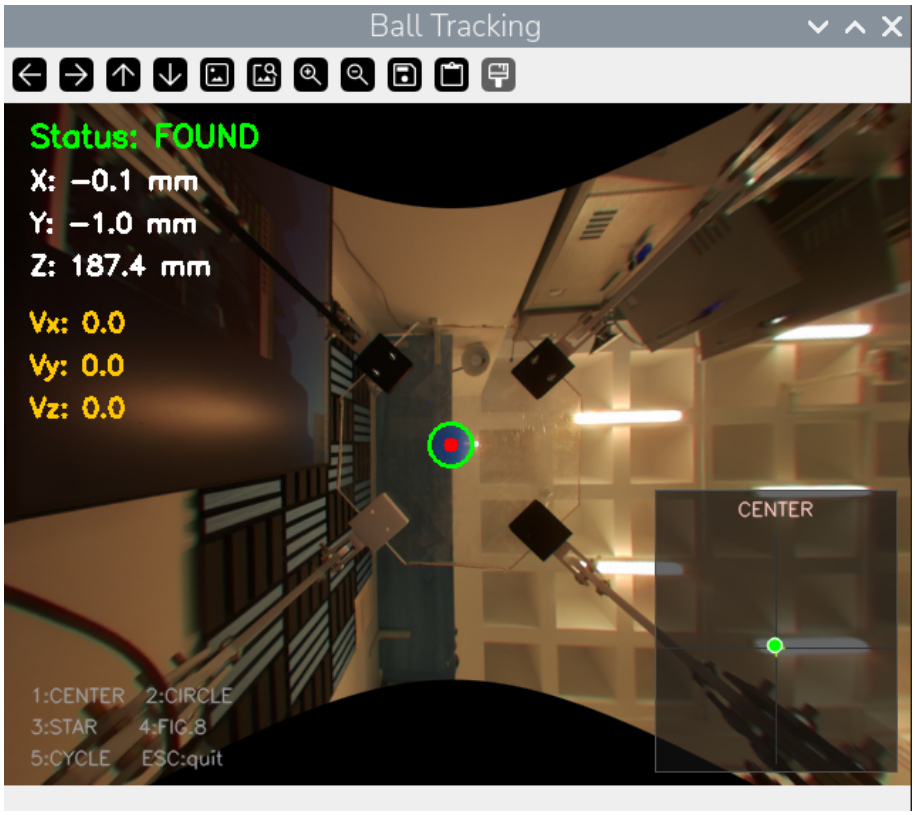


Figure 4.2: *The GUI interface in CENTER mode: the ball is kept in the center of the platform, and the minimap indicates the target position (green) relative to the actual position of the ball (yellow).*

4.3.7 Trajectory Generator and Pattern Selection

In addition to stabilization at a fixed point, the **OctoBouncer** system implements a trajectory generator that forces the ball to follow predefined geometric shapes. The active mode is controlled bidirectionally through the UART protocol: the Teensy 4.1 notifies the Raspberry Pi at each pattern transition, and the user can change the mode from the graphical interface by pressing the keys `1`–`5`, without any need for a direct modification in the code.

Four distinct trajectories are defined, plus an automatic cycling mode:

- **CENTER (key 1):** The ball is kept at the origin of the platform. The target is fixed at $(0, 0)$.
- **CIRCLE (key 2):** The ball follows a circular trajectory with a radius of 50 mm and a period of 5 s.
- **STAR (key 3):** A 5-pointed star-shaped trajectory, alternating between the outer radius of 60 mm and the inner one of 25 mm, with a period of 10 s.
- **FIGURE 8 (key 4):** A figure-eight-type trajectory (lemniscate), with a horizontal semi-axis of 60 mm and a vertical one of 30 mm, with a period of 6 s.
- **CYCLE ALL (key 5):** The system automatically cycles through all four modes above, 25 s each, with smooth transitions ensured by a target rate limiter of 120 mm/s.

The synchronization of the visualization with the real execution is ensured by transmitting the current target position (t_x, t_y) from the Teensy to the Raspberry Pi through a dedicated `PACKET_TYPE_TARGET_POS` packet at every 5 ms iteration. Thus, the green dot in the animation reflects exactly the coordinate followed by the PID controller.

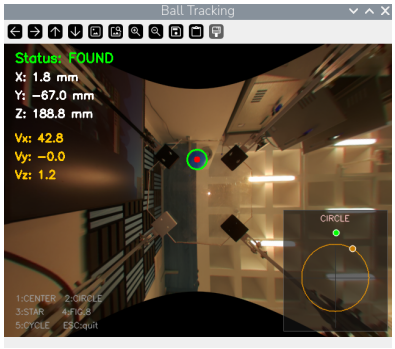


Figure 4.3: *CIRCLE mode.*

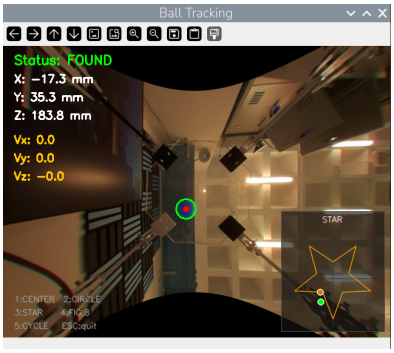


Figure 4.4: *STAR mode.*

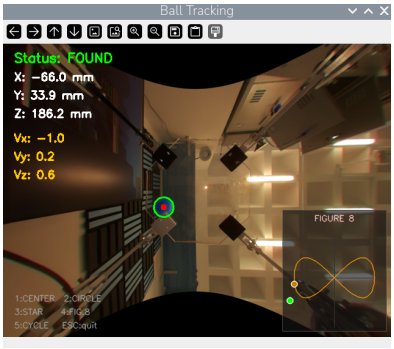


Figure 4.5: *FIG8 mode.*

4.3.8 Real-Time PID Tuning Interface

The `4_gui_track_pid.py` script extends the diagnostic interface with three sliders that allow adjusting the coefficients K_p, K_i and K_d in real time, without restarting the system. The sliders start from the default values hardcoded in the firmware ($K_p = 0.149$, $K_i = 0.112$, $K_d = 0.068$, for positions expressed in the units of the HSV pipeline; see Section 5.1). The values are scaled internally by the factor $\frac{1}{1000}$ and are transmitted to the Teensy through a dedicated packet of type `PACKET_TYPE_PID_UPDATE` once the user modifies any slider. The current coefficients are also displayed in the video overlay (in light green, under the velocities section).

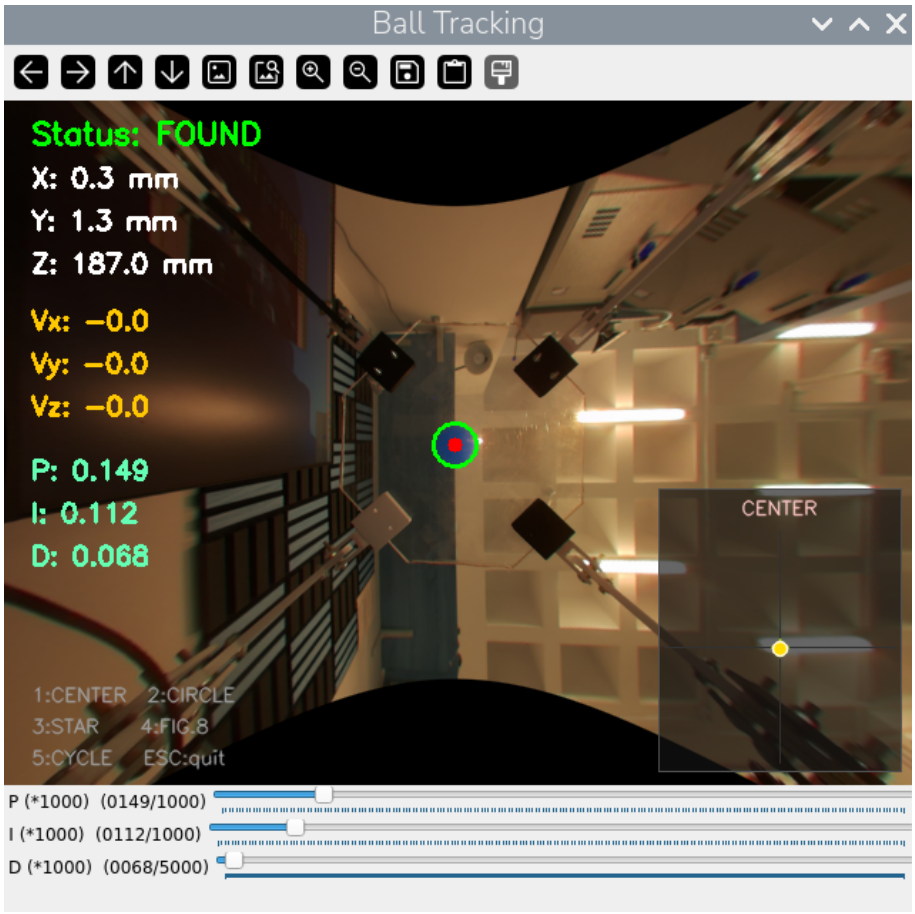


Figure 4.6: *The PID tuning interface: the P, I, D sliders visible in the lower part of the window, with the current values displayed in the overlay (light green). Pattern selection remains available through the keys 1–5.*

4.3.9 Serial Communication Protocol

At the end of each processing cycle, the ball's state vector is serialized and transmitted through **UART** at **115,200 baud**. The implemented custom protocol has the structure:

$$\underbrace{\texttt{0xAA}}_{\text{start}} \mid \underbrace{\texttt{TYPE}}_{\text{packet type}} \mid \underbrace{\texttt{SIZE}}_{\text{length}} \mid \underbrace{\texttt{PAYLOAD}}_{\text{data}} \mid \underbrace{\texttt{CSUM}}_{\text{XOR}} \mid \underbrace{\texttt{0xBB}}_{\text{stop}} \tag{4.4}$$

The checksum is calculated as an accumulated XOR over the packet type, the size and each byte of the payload. The checksum is validated on the Teensy before the data is introduced into the PID loop; a corrupted packet is ignored, and the system continues with the last valid state. Five packet types are defined: `BALL_DATA` (the positions X, Y, Z and the velocities V_x, V_y, V_z packed as 6 32-bit floats, with a status byte `is_found`), `PID_UPDATE` (three floats with the new coefficients), `START` (initialization signal), `PATTERN_CHANGE` (mode change, bidirectional) and `TARGET_POS` (the current target position and the height setting of the platform, transmitted by the Teensy to the Pi at every 5 ms iteration for the synchronization of the minimap). The Teensy reads only the position and the detection status from `BALL_DATA`.

4.4 Data Filtering (EMA Filter)

The received position data may exhibit white noise due to the camera resolution or lighting variations. To normalize the system's response without introducing significant latencies, we used an **EMA (Exponential Moving Average)** filter [19]:

$$X_{filtrat} = \alpha \cdot X_{raw} + (1 - \alpha) \cdot X_{anterior} \tag{4.5}$$

The value $\alpha = 0.15$ was chosen experimentally to balance noise attenuation with the reaction speed of the system. The target position is passed through an identical filter before its velocity is computed, so that the two velocities compared in the derivative term have the same delay.

4.5 Real-Time Management and Synchronization

The stability of a "Ball and Plate" system depends on the constancy of the sampling period. The Teensy 4.1 uses a high-precision internal clock to ensure the execution of the control loop at exactly 5 ms. The firmware is written in C++ and built with PlatformIO [18].

- If the data from the Raspberry Pi is delayed, the system uses the last known state to maintain stability.
- The motor movement is interpolated to avoid mechanical shocks, ensuring a smooth transition between the calculation steps of the PID.

4.6 Safety Limitations (Software Constraints)

To protect the mechanical structure, the angles calculated by the PID are passed through a limiting block before being converted into arm commands (Section 3.3), and every arm command is limited to its safe range. This prevents reaching the mechanical limits of the joints in the case of violent movements or if the ball accidentally leaves the platform.

4.7 Plate Height Control

The arm angles are measured from the mechanical low stop, so the arms must rest on the stop when the Raspberry Pi sends the `START` command. At startup the Teensy lifts all arms to 5° in 1 s, then raises them together at 2°/s to 15° while the balancing loop is already active, so the ball can be placed on the platform immediately.

From there, the common height angle α_h is controlled from the measured depth of the ball: since the camera is under the platform, a higher platform increases Z. The Teensy keeps the ball at $Z = 195 \pm 4$ mm by moving α_h at 0.05°/s per millimeter of error, at most 2°/s, within the range 15°–30°. The depth is low-pass filtered and only used while the ball is within 75 mm of the center, where its estimate is reliable. The height setting is reported to the Raspberry Pi in the `TARGET_POS` packet.

Chapter 5

Experimental Results and Conclusions

5.1 Experimental Method

We measured the tracking performance of the **OctoBouncer** on the physical system in its final configuration: **median-filtered depth**, a **derivative term acting on the velocity of the ball relative to the target**, and **closed-loop control of the plate height**. Four runs (runs 7–10, recorded on 24 October 2026) were made in the automatic cycling mode (**CYCLE ALL**), in which the controller follows each pattern for 25 s: CENTER, CIRCLE, STAR and FIGURE 8. The ball was placed on the plate at start-up and the system was not touched afterwards. A fifth run (run 1), recorded with the same vision system but before the control changes (derivative term on the ball position only, depth not filtered), serves as the reference for the comparison in Section 5.4.

In these runs the ball positions are expressed in true millimeters. The HSV pipeline of Section 4.3 reports about 1/1.4 of the true distances, because the dilated mask overestimates the apparent radius of the ball; the PID gains of Chapter 4 were therefore divided by the same factor 1.4, so that the controller behaves identically.

For every camera frame (**60 Hz**) the tracking script on the Raspberry Pi 5 logs the measured ball position $\mathbf{p}_{ball}(t)$ and the target $\mathbf{p}_{target}(t)$ that the Teensy 4.1 reports through the `TARGET_POS` packet every 5 ms, i.e. exactly the set-point used by the PID controller. All coordinates are in millimeters in the camera frame. The tracking error is defined as:

$$e(t) = \|\mathbf{p}_{ball}(t) - \mathbf{p}_{target}(t)\| \tag{5.1}$$

The following quantities were computed for each pattern phase:

- **Steady-state error:** The mean, RMS, 95th percentile (P95) and maximum of $e(t)$ over the last 15 s of each 25 s phase. The final 0.5 s of each phase are excluded, because the target already moves towards the next pattern.
- **Settling time (t_s):** The time after the pattern switch from which the 1 s moving RMS of $e(t)$ stays below 1.1 times the largest 1 s moving RMS observed in the steady-state window. A value of zero means that the switch produced no error transient larger than the pattern's own steady-state error.
- **Lag:** The time shift of the measured path that minimizes its mean squared distance to the target (positive: the ball is behind the target; negative: ahead of it).
- **Shape deviation and amplitude ratio:** The mean distance of the measured path from the ideal curve, and the ratio between the ball's mean distance from the center and the target's.

Two checks support the validity of the measurement. First, the commanded target coincides with the mathematical trajectories of Section 5.2 (maximum deviation 0.07 mm), so the target rate limiter is not active in steady state and the comparison with the ideal curves is exact. Second, while the ball is held at the center, the high-frequency scatter of the measured position (deviation from its 0.25 s moving average) is 0.57 ± 0.12 mm RMS, an order of magnitude below the tracking errors reported below. The ball path is measured by the same vision system that closes the control loop; no independent ground-truth sensor was available.

5.2 Trajectory Tracking

The target trajectories are defined in the firmware of the Teensy 4.1 as:

$$\text{CIRCLE:} \quad x_t = 50\cos(\omega t),\ y_t = 50\sin(\omega t), \qquad \omega = 2\pi/5 \text{ rad/s} \tag{5.2}$$

$$\text{FIGURE 8:} \quad x_t = 60\sin(\omega t),\ y_t = 30\sin(2\omega t), \qquad \omega = 2\pi/6 \text{ rad/s} \tag{5.3}$$

and, for the STAR, as straight segments between ten vertices at the angles $\varphi_i = \pi/2 + i\pi/5$ with radii alternating between 60 mm and 25 mm, covering one vertex per second (period 10 s). All values are in millimeters.

Figure 5.1 compares these curves with the measured ball paths in steady state, and Figure 5.2 shows the same data as time series.

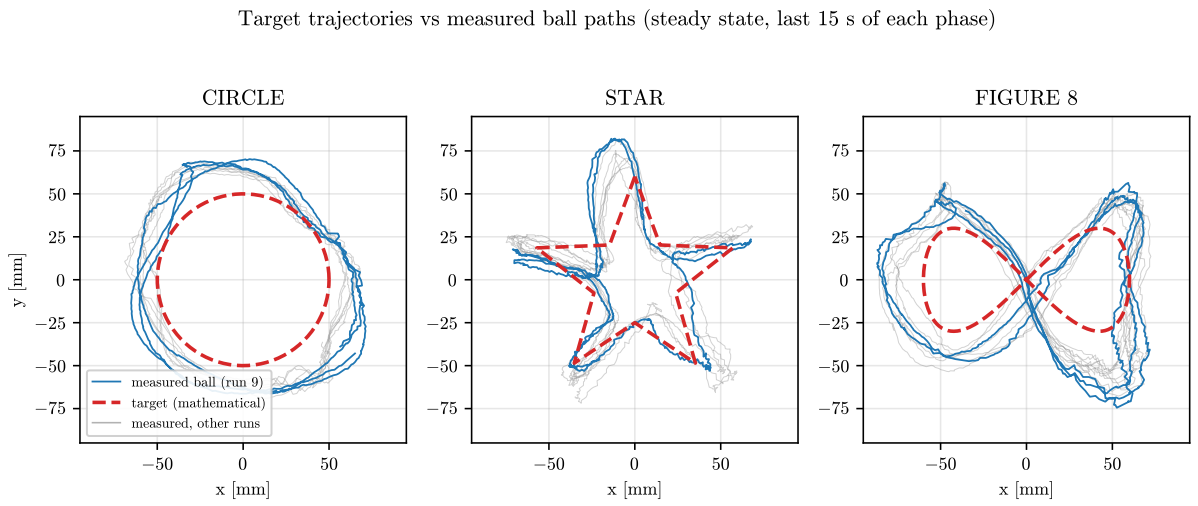


Figure 5.1: *Target trajectories (red, dashed) and measured ball paths in steady state (last 15 s of each phase). Blue: run 9; gray: runs 7, 8 and 10.*

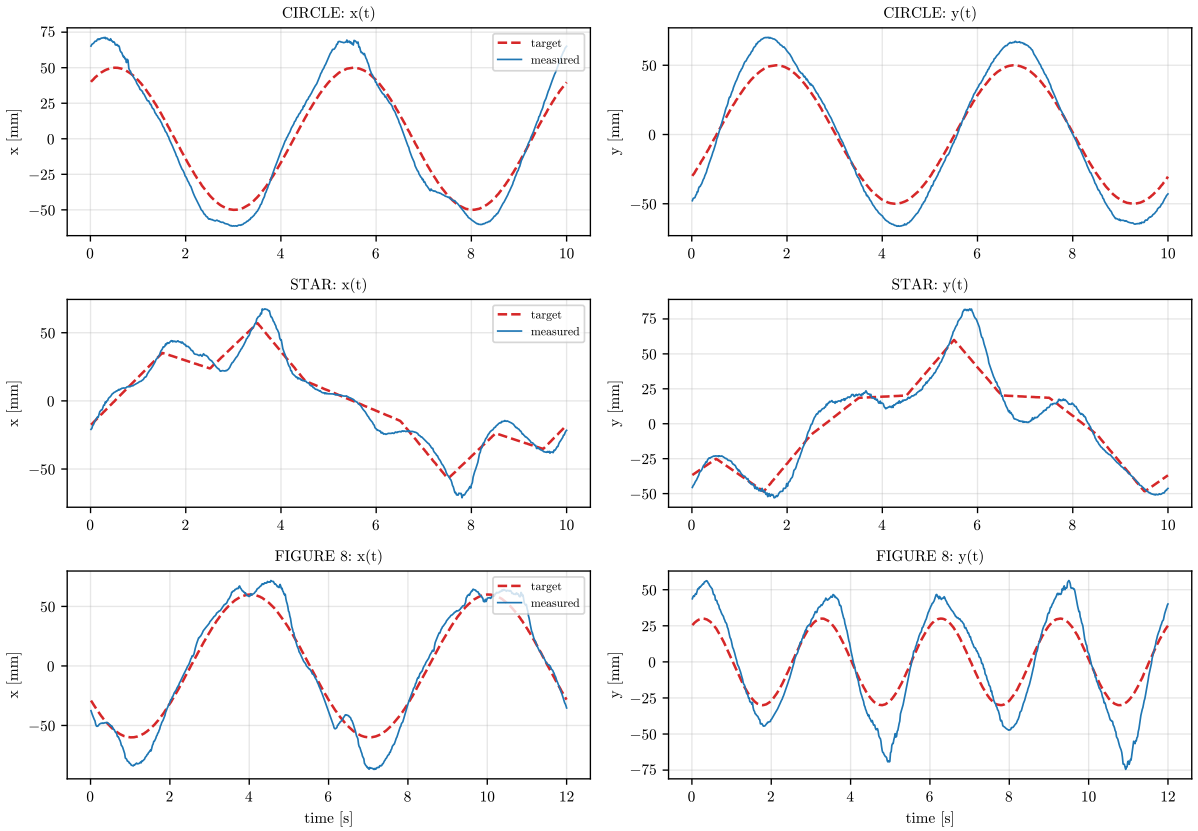


Figure 5.2: *Target (red, dashed) and measured (blue) coordinates over time, run 9, steady state.*

The ball follows all three shapes with **almost no time lag** (Table 5.1): 0.05 s on the figure 8 and 0.10 s on the star, while on the circle it is even 0.04 s *ahead* of the target (negative lag). This is a consequence of the derivative term acting on the velocity relative to the target: it reacts to the motion of the target itself and produces a phase lead, which compensates the delay of the plate and of the ball. The price of this lead is an overshoot in amplitude, so the remaining error is mostly geometric: the ball travels on a path that is larger than the target, by 25 % on the circle and 31 % on the figure 8, overshooting outwards where the curvature is highest. The star, made of straight segments covered at low speed, is followed most faithfully: its measured path stays on average 7.4 mm from the ideal shape and is only 9 % larger, the extra displacement appearing mainly at the outer vertices.

Pattern	**Lag [s]**	**RMS without lag [mm]**	**Shape deviation [mm]**	**Amplitude ratio**
CIRCLE	-0.04 ± 0.01	15.0 ± 1.6	12.8 ± 1.6	1.25 ± 0.03
STAR	0.10 ± 0.02	13.5 ± 0.7	7.4 ± 0.6	1.09 ± 0.01
FIGURE 8	0.05 ± 0.02	18.5 ± 1.4	13.7 ± 1.1	1.31 ± 0.03

Table 5.1: *Decomposition of the tracking error on the moving patterns: time lag, error without the lag, deviation from the ideal shape and amplitude ratio.*

5.3 Settling Time and Steady-State Error

Table 5.2 summarizes the settling time and the steady-state tracking error. Figure 5.3 shows the error over a complete pattern cycle, and Figure 5.4 the behavior after switching to CENTER.

Pattern	**Runs**	**Settling [s]**	**Mean [mm]**	**RMS [mm]**	**P95 [mm]**	**Max [mm]**
CENTER	3	4.6 ± 4.1	2.9 ± 1.5	3.4 ± 1.8	7.0 ± 4.0	8.4 ± 4.2
CIRCLE	3	2.2 ± 0.7	14.2 ± 1.5	15.3 ± 1.5	24.3 ± 1.2	29.1 ± 1.9
STAR	4	2.4 ± 1.8	12.5 ± 0.9	14.4 ± 1.0	26.7 ± 2.8	33.8 ± 3.4
FIGURE 8	4	0	17.1 ± 1.2	18.9 ± 1.5	30.6 ± 5.3	41.7 ± 6.5

Table 5.2: *Settling time and steady-state tracking error of the final configuration (mean ± standard deviation over the runs).*

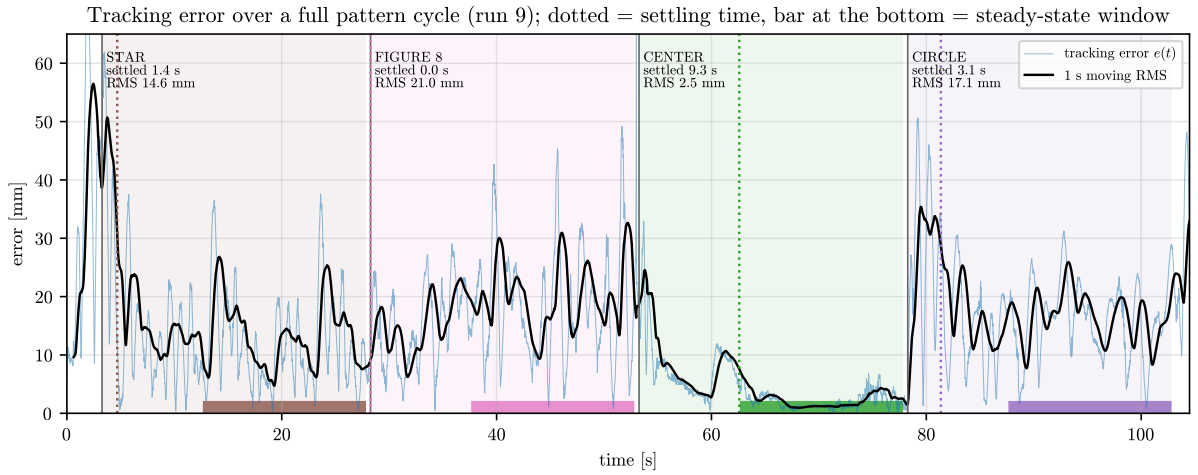


Figure 5.3: *Tracking error over one complete pattern cycle (run 9). Thin line: $e(t)$; thick line: 1 s moving RMS; dotted lines: settling time of each phase; colored bars at the bottom: steady-state window.*

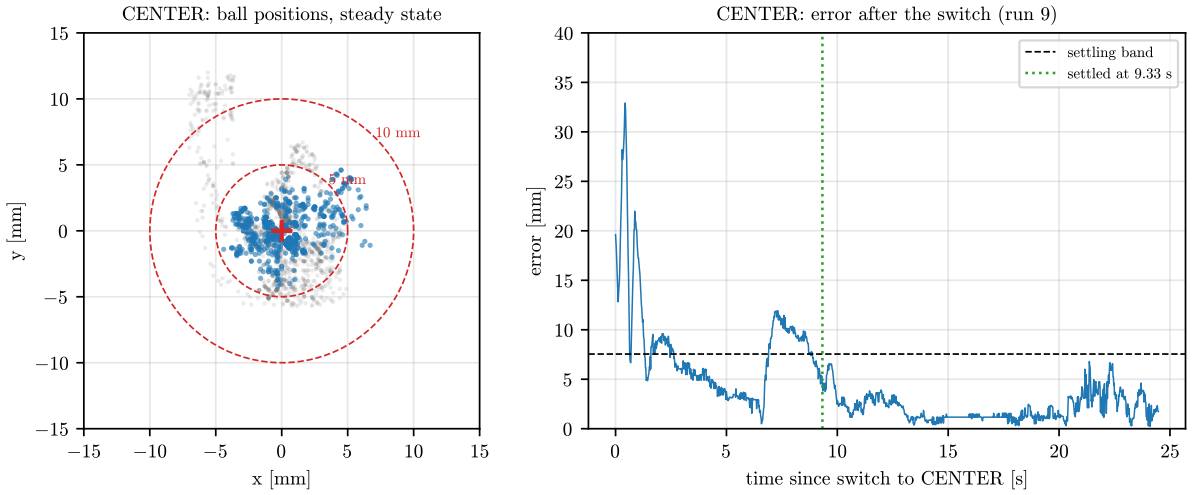


Figure 5.4: *CENTER mode. Left: ball positions in steady state (blue: run 9, gray: runs 7 and 8). Right: tracking error after the switch to CENTER in run 9, with the settling band.*

When the target is fixed at the center, the ball is held with a **mean error of** 2.9 ± 1.5 **mm**. In steady state it stays within 5 mm of the center in 97 % and 96 % of the samples in runs 8 and 9, and in 65 % in run 7, where it stays within 10 mm in 90 % of the samples. The transition into CENTER (the target returns from the previous pattern at the 120 mm/s rate limit) settles in 2.1 s and 2.5 s in runs 7 and 8. In run 9 the settling time is 9.3 s, because of a single excursion of about 12 mm, 7 s after the switch (Figure 5.4, right), whose cause was not identified; for this reason the per-run values are listed in Table 5.3 in addition to the mean. Switching to CIRCLE or STAR settles within 2.2 ± 0.7 s and 2.4 ± 1.8 s. Switching from STAR to FIGURE 8 produced no transient beyond the figure 8's own steady-state error in any of the four runs.

On the moving patterns the **steady-state error is 12.5 mm (star), 14.2 mm (circle) and 17.1 mm (figure 8)** on average, with 95 % of the samples below 26.7, 24.3 and 30.6 mm respectively. The spread between runs is small (standard deviation 0.9–1.5 mm on the mean error), which indicates that these values are repeatable. The ball was detected in 100 % of the analyzed frames.

Run	Pattern	Settling [s]	Mean [mm]	RMS [mm]	P95 [mm]	Max [mm]
7	CENTER	2.08	4.7	5.5	11.6	13.2
7	CIRCLE	1.78	13.5	14.5	23.3	27.8
7	STAR	5.05	11.6	13.2	23.4	29.7
7	FIGURE 8	0.00	15.8	17.4	26.5	33.6
8	CENTER	2.45	1.9	2.4	4.7	5.3
8	CIRCLE	1.85	13.2	14.4	23.8	28.4
8	STAR	1.75	13.8	15.5	27.9	32.4
8	FIGURE 8	0.00	17.4	19.0	28.4	40.9
9	CENTER	9.33	2.1	2.5	4.7	6.8
9	CIRCLE	3.08	16.0	17.1	25.7	31.3
9	STAR	1.41	12.2	14.6	29.9	37.6
9	FIGURE 8	0.00	18.6	21.0	38.4	49.2
10	STAR	1.50	12.4	14.1	25.6	35.4
10	FIGURE 8	0.00	16.5	18.2	29.1	43.3

Table 5.3: *Settling time and steady-state tracking error for each run.*

5.4 Effect of the Control Changes

Table 5.4 and Figure 5.5 compare the final configuration with run 1. Before the changes, the derivative term reacted to the ball's own velocity, and therefore also braked a ball that was correctly following a moving target. The ball lagged the target by 0.37–0.43 s and its path was 25–48 % larger than the target. Making the derivative act on the velocity relative to the target, and filtering the depth used to scale the position, removed most of this lag and **reduced the mean tracking error by 36–61 %** on the moving patterns and by 34 % at the center.

Pattern	Before: mean / P95 [mm]	After: mean / P95 [mm]	Mean error reduction
CIRCLE	36.3 / 44.9	14.2 / 24.3	61 %
STAR	19.7 / 41.4	12.5 / 26.7	36 %
FIGURE 8	36.7 / 55.4	17.1 / 30.6	53 %
CENTER	4.4 / 7.6	2.9 / 7.0	34 %

Table 5.4: *Steady-state tracking error before (run 1) and after (runs 7–10) the control changes.*

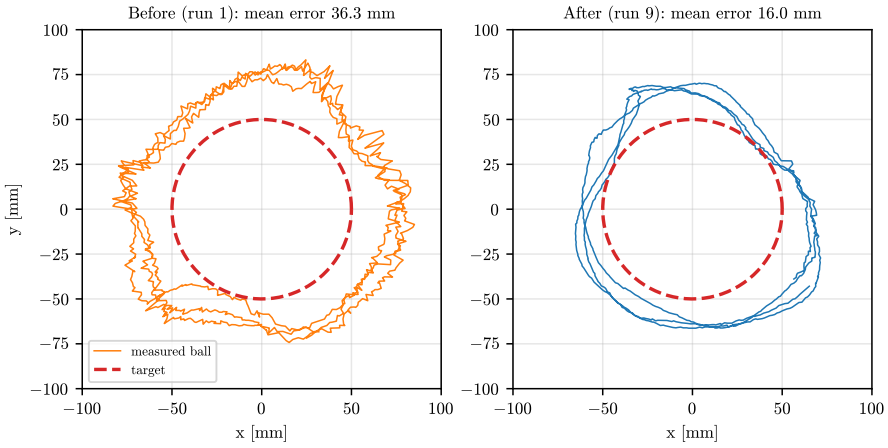


Figure 5.5: *CIRCLE in steady state before (run 1) and after (run 9) the control changes.*

5.5 Discussion and Limitations

The measurements show that the system stabilizes the ball at a fixed point with an error of a few millimeters and follows the three trajectories with a repeatable error of 12–17 mm, which is small compared with the size of the trajectories (60 mm) and of the area in which the ball can move (about 117 mm from the center of the platform). The remaining error is dominated by the outward deviation on curved paths rather than by lag. Reducing it would probably require a *feed-forward* term derived from the target's acceleration, or a model-based controller such as the LQR or MPC approaches proposed in the future developments; this was not tested.

The results have the following limitations:

- **Number of runs:** Three to four runs per pattern, recorded in one session and one lighting condition; the standard deviations describe the repeatability between these runs only.
- **Fixed transitions:** Each phase is preceded by a fixed pattern (the cycling order), so the settling times characterize those particular transitions.
- **Single sensor:** The ball position is measured by the same camera that closes the loop. Systematic errors of the vision system (for example in the depth scale) would affect the target and the measurement in the same way and are not visible in these metrics.

5.6 Industrial and Commercial Applications

The technology developed in **OctoBouncer** is not limited to balancing a ball. Its core is the combination of three elements that are generally useful in automation: **contact-free measurement** of an object's position on a surface with a single calibrated camera, **real-time control of the tilt and height** of a platform driven by a parallel mechanism, and **trajectory tracking** with a closed feedback loop. The measurements in Section 5.3 show what this combination achieves on low-cost hardware: an object held at a fixed point within a few millimeters, and predefined paths followed with a repeatable error of

12–17 mm, with vision at 60 Hz and control at 200 Hz. These capabilities can be applied directly in the following real-world fields:

- **Stabilized transport platforms:** Service robots, automated guided vehicles (AGVs) and delivery systems that carry liquids or fragile goods need a platform that tilts against the accelerations of the vehicle, so that the load does not slide or spill. The inverse kinematics and the height control of OctoBouncer solve exactly this problem; the camera would be complemented by an inertial sensor measuring the motion of the base.
- **Motion simulators and test benches:** Platforms with controlled pitch, roll and height are the basis of driving and flight simulators and of test benches that expose products or sensors (for example inertial measurement units) to defined movements. The four-arm parallel mechanism offers these three degrees of freedom with simpler mechanics than a six-actuator Stewart platform [14], and the trajectory generator can reproduce defined test movements.
- **Material handling without grippers:** In manufacturing, small parts can be positioned, aligned or sorted by tilting the surface they lie on, instead of being picked up. The vision-guided control of OctoBouncer moves a free object along a prescribed path in exactly this way, which applies to part feeders, sorting tables and the alignment of components before assembly.
- **Leveling and orientation systems:** Work tables, optical benches, antennas and solar panels have to be kept level or oriented in a defined direction. The same control structure (measurement, PID controller, inverse kinematics) keeps a platform at a set orientation and height, with the camera replaced by the sensor relevant for the application.
- **Medical rehabilitation and balance training:** Tilting platforms are used in physiotherapy for ankle rehabilitation and balance training. Programmable trajectories such as CIRCLE or FIGURE 8 correspond to the controlled movement patterns used in these exercises, and the height control makes it possible to adapt the platform to the patient.
- **Education and research:** Ball-and-plate systems are established laboratory equipment in control engineering education [1]. OctoBouncer can be used as a teaching platform covering image processing, inverse kinematics, PID tuning and real-time embedded programming on open hardware, at the cost given in the bill of materials.

In all these applications the system would keep its architecture: a high-level processor for perception, a real-time microcontroller for control, and a parallel mechanism for actuation. The main adaptations would be the sensor (camera, inertial sensor or distance sensor, depending on the task), the size and load capacity of the mechanism, and the safety certification required in an industrial or medical environment.

5.7 Conclusions and Future Developments

The **OctoBouncer** project successfully demonstrated the convergence between mechatronics and automatic control. Although the current PET-G prototype is perfectly

functional for stabilization tasks (Ball and Plate), the rigidity of the plastic limits the frequency of violent movements.

For the future, we propose the following development directions:

1. **Change of Material:** CNC machining of the arms from aluminum to maximize rigidity and to activate the full "bouncing" function (the rhythmic throwing and catching of the ball).

2. **Complex Trajectories:** Implementing trajectory generators through direct input from the user.

3. **Advanced Control:** Exploring LQR (Linear-Quadratic Regulator) or Model Predictive Control (MPC) algorithms to replace the classic PID.

Bibliography

[1] W. Yu, F. Ortiz, "Optimized Fuzzy Control of a Ball and Plate System," *Proceedings of the IEEE International Symposium on Intelligent Control*, pp. 7–12, 2004.

[2] Electron Dust, The Octo-Bouncer https://www.electrondust.com/2020/03/01/the-octo-bouncer/

[3] Raspberry Pi Foundation, *Raspberry Pi 5 Product Brief*, Raspberry Pi Foundation, 2023. https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008348-DS-6-raspberry-pi-5-product-brief.pdf

[4] PJRC, *Teensy 4.1 Technical Specifications*, PJRC Electronics, 2020. https://www.pjrc.com/store/teensy41.html

[5] Leadshine Technology, *DM556 Digital Stepper Drive User Manual*, Leadshine Technology Co., Ltd., 2015. https://www.leadshine.com

[6] Robofun, *NEMA 17 Stepper Motor, 1.5 A, 200 Steps, with Cable*, https://www.robofun.ro/motor-stepper-nema17-1-5a-200-pasi-cu-cablu.html

[7] G. Bradski, A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*, O'Reilly Media, 2008.

[8] J. Kannala, S. S. Brandt, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 6, pp. 1335–1340, Jun. 2006.

[9] OpenCV Development Team, *OpenCV 4.x Documentation: Camera Calibration and 3D Reconstruction*, 2024. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

[10] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[11] K. J. Åström, T. Hägglund, *PID Controllers: Theory, Design and Tuning*, 2nd ed., Instrument Society of America, 1995.

[12] K. J. Åström, B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, 3rd ed., Prentice Hall, 1997.

[13] J. P. Merlet, *Parallel Robots*, 2nd ed., Springer, 2006.

[14] D. Stewart, "A Platform with Six Degrees of Freedom," *Proceedings of the Institution of Mechanical Engineers*, vol. 180, no. 1, pp. 371–386, 1965.

[15] M. Galeja et al., "Mechanical Properties of 3D-Printed PETG Polymer," *Polymers*, vol. 12, no. 12, art. 2959, 2020.

[16] Ontario Tech University, *Camera Calibration in OpenCV* https://csundergrad.science.uoit.ca/courses/cv-notes/notebooks/02-camera-calibration.html

[17] Raspberry Pi Foundation, *Picamera2 Library Documentation*, 2023. https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf

[18] PlatformIO Labs, *PlatformIO IDE and Build System Documentation*. https://docs.platformio.org

[19] G. Hunter, *Exponential Moving Average (EMA) Filter*, 2024. https://blog.mbedded.ninja/programming/signal-processing/digital-filters/exponential-moving-average-ema-filter/